Detecting Fraudulent Transactions Using Machine Learning: A Supervised Learning Approach

1. Problem Statement

Fraudulent transactions cost businesses billions annually. The goal of this project is to build a machine learning model that can accurately classify whether a transaction is fraudulent based on behavioral and transactional features.

2. Dataset Overview

The dataset includes 150,000 e-commerce transactions with features such as user Id, sign up date, transaction date and time, transaction amount, device Id, store, browser, sex, age and IP location. The target variable is a binary indicator of whether the transaction was fraudulent.

3. Tools and Techniques Used

- Languages: Python
- Libraries: pandas, NumPy, scikit-learn, XGBoost, matplotlib, seaborn, joblib
- Environment: Jupyter Notebook, Anaconda
- Deployment: Flask API and AWS EC2 instance

4. Data Preprocessing

- Mapping IP addresses to Countries using GeoLite2 IP geolocation database
- No missing values in the dataset
- Categorical variables encoded (one-hot and frequency encoding)
- Feature scaling using StandardScaler
- Outlier checking

5. Exploratory Data Analysis

- Target variable distribution (The dataset is imbalanced)
- No significant difference in amount variability between Fraudulent and legitimate transactions
- Fraudulent transactions are concentrated in specific countries
- Fraud rates vary across browsers and store categories
- Correlation analysis helped detecting multicollinearity

6. Model Building

- Baseline Models: Logistic Regression
- Advanced Models: Random Forest, XGBoost
- Deep Learning: Autoencoder (unsupervised anomaly detection)
- Used GridSearchCV for hyperparameter tuning and StratifiedKFold for evaluation

7. Evaluation Metrics

Model	Precision	Recall	AUC-ROC
Logistic Reg.	0.00	0.00	0.51
Random Forest	1.00	0.53	0.77
XGBoost	0.98	0.53	0.77
Autoencoder	0.14	0.08	0.63

Random Forest and XGBoost performed best overall. Autoencoder was useful as an anomaly detector but had lower performance.

8. Key Takeaways

- XGBoost and Random Forest provided the highest fraud detection rate with minimal false positives
- Feature importance analysis helped explain model behavior to stakeholders
- SHAP values gave more interpretable, instance-level explanations
- Demonstrated feasibility of deploying ML for real-time fraud scoring

9. Resources

GitHub Repo

Kaggle Notebook

Flask API Demo